مقایسهٔ ساخت عادی و چندمرحله‌ای Docker

موضوع و محدوده

این گزارش برای پروژهٔ FastAPI مدیریت هزینه‌ها تهیه شده است. نسخهٔ قبلی پروژه Dockerfile نداشت؛ در این تمرین ساختار Docker و CI/CD از ابتدا اضافه شد. منظور از ساخت عادی در این متن، Dockerfile تک‌مرحله‌ای است و نباید آن را با نصب دستی برنامه روی سرور یکسان دانست.

ساخت تک‌مرحله‌ای

در روش تک‌مرحله‌ای نصب وابستگی، آماده‌سازی فایل‌ها و اجرای برنامه در یک مرحله انجام می‌شوند. نوشتن آن ساده است و برای برنامهٔ کوچک با وابستگی‌های آماده می‌تواند انتخاب مناسبی باشد. اگر ابزارهای کامپایل یا تست در همان مرحله نصب شوند و در لایه‌های قبلی باقی بمانند، image نهایی آن‌ها را هم همراه خواهد داشت. حذف فایل در یک لایهٔ بعدی الزاماً حجم لایهٔ قبلی را کم نمی‌کند. [۱، ۲]

ساخت چندمرحله‌ای

هر FROM یک مرحلهٔ مستقل ایجاد می‌کند. با COPY --from فقط خروجی لازم از مرحلهٔ قبلی وارد مرحلهٔ اجرای نهایی می‌شود. برای این پروژه یک مرحله محیط مجازی وابستگی‌ها را آماده می‌کند، یک مرحله PO را به MO تبدیل می‌کند و مرحلهٔ آخر فقط برنامه و وابستگی‌های اجرایی را تحویل می‌دهد. polib و ابزارهای تست در محیط اجرایی لازم نیستند. [۱]

تفاوت عملی و هزینهٔ نگهداری

روش چندمرحله‌ای مرز ابزار ساخت و ابزار اجرا را روشن می‌کند، اما تعداد مراحل و نیاز به هماهنگی نسخهٔ Python و کتابخانه‌های باینری بیشتر می‌شود. برای کپی محیط مجازی باید مسیر و سازگاری سیستم‌عامل مراحل حفظ شوند. چندمرحله‌ای بودن به‌تنهایی تضمین کاهش حجم یا افزایش سرعت نیست؛ اگر مرحلهٔ اول و آخر تقریباً محتویات یکسان داشته باشند، تفاوت حجم ممکن است اندک باشد. در این گزارش درصد کاهش حجم یا سرعت ادعا نشده، چون benchmark مقایسه‌ای انجام نشده است.

انتخاب مناسب

برای این پروژه چندمرحله‌ای مناسب است، چون ابزار ترجمه فقط هنگام ساخت لازم است و وابستگی‌های تست باید بیرون از runtime بمانند. برای یک اسکریپت بسیار کوچک و بدون ابزار ساخت، تک‌مرحله‌ای می‌تواند ساده‌تر باشد. تعداد مراحل هدف نیست؛ خروجی قابل نگهداری و قابل آزمایش هدف است.

بهبودهای Dockerfile پروژه

پایه و وابستگی‌ها

پایهٔ رسمی Python 3.12 slim-bookworm با digest ثابت انتخاب شده تا تغییر ناخواستهٔ tag پایه، محتوای ساخت را عوض نکند. نسخهٔ وابستگی‌های مستقیم pin شده است؛ وابستگی‌های انتقالی هنوز lock کامل ندارند، پس تکرارپذیری صددرصد ادعا نمی‌شود. نصب فقط wheel مجاز است؛ اگر برای یک پلتفرم wheel موجود نباشد، ساخت متوقف می‌شود و به‌صورت پنهان کامپایل انجام نمی‌شود.

ترتیب لایه‌ها و محتوای image

ابتدا فایل requirements کپی و وابستگی‌ها نصب می‌شوند؛ تغییر کد بدون تغییر وابستگی‌ها می‌تواند از cache استفاده کند. .dockerignore با فهرست مجاز، فایل‌های Git، دیتابیس، گواهی، تست‌ها و اسناد را از context خارج می‌کند. فایل‌های برنامه نیز صریح کپی می‌شوند. cache مربوط به pip نگه‌داری نمی‌شود و فایل‌های ترجمه در مرحلهٔ مجزا کامپایل می‌شوند. [۲]

اجرای محدود و دادهٔ ماندگار

کاربر اجرایی UID=10001 است. مسیر /data برای SQLite قابل نوشتن است و compose برای آن volume می‌سازد. فایل‌سیستم اصلی read-only است، قابلیت‌های اضافی Linux حذف می‌شوند و no-new-privileges فعال است. کلید JWT فقط هنگام اجرا از محیط دریافت می‌شود؛ در ARG، لایهٔ image یا repository ذخیره نمی‌شود.

آمادگی و migration

GET /health اتصال و وجود جدول هزینه را بررسی می‌کند. HEALTHCHECK از کتابخانهٔ استاندارد Python استفاده می‌کند تا curl فقط برای بررسی سلامت نصب نشود. migration در دستور شروع عادی API اجرا نمی‌شود؛ ابتدا یک job یا دستور جداگانه اجرا می‌شود تا چند replica همزمان schema را تغییر ندهند. SQLite این تمرین برای یک instance طراحی شده است.

HTTPS و مرز امنیت

Uvicorn داخل شبکهٔ کانتینر HTTP سرویس می‌دهد و compose آن را فقط روی loopback میزبان منتشر می‌کند. برای استفادهٔ مرورگر باید reverse proxy با HTTPS و مبدأ APP_ORIGIN درست در جلو قرار گیرد؛ Secure بودن کوکی خاموش نمی‌شود. فایل Compose ارائه‌شده به‌تنهایی گواهی TLS تهیه نمی‌کند و استقرار عمومی را انجام نمی‌دهد.

زنجیرهٔ CI/CD و نتیجهٔ بررسی

CI: آزمون قبل از ساخت

workflow در push به main و شاخهٔ تمرین، tagهای v و pull request اجرا می‌شود. job اول Python را آماده می‌کند، ترجمه‌ها را کامپایل و با MO ثبت‌شده مقایسه می‌کند و تست‌های API، امنیت، migration و زبان را اجرا می‌کند. job ساخت فقط پس از موفقیت تست‌ها اجرا می‌شود. اکشن‌ها با commit SHA ثابت ارجاع داده شده‌اند.

ساخت و تحویل پیوسته

job دوم image را می‌سازد و یک کانتینر موقت با دیتابیس موقت اجرا می‌کند. سپس همان image را به artifact تبدیل می‌کند. job انتشار همان artifact را بارگذاری و با tag شامل SHA کامل commit به GHCR می‌فرستد؛ image دوباره ساخته نمی‌شود. pull request اجازهٔ انتشار ندارد. مجوز packages:write فقط برای job انتشار و GITHUB_TOKEN کوتاه‌عمر در نظر گرفته شده است. [۳، ۴]

تحویل image در برابر استقرار روی سرور

Continuous Delivery در این تمرین یعنی تحویل خودکار image آزمایش‌شده به registry. Continuous Deployment به سرور نیازمند مقصد، شبکه، secretهای اجرا، backup، HTTPS و برنامهٔ rollback است. چون مقصد سرور تعیین نشده، workflow ادعای استقرار سرویس عمومی ندارد. برای ارتقا باید image مشخص را pull، از SQLite پشتیبان تهیه، migration را کنترل‌شده اجرا و سلامت سرویس را بررسی کرد. rollback image بدون توجه به سازگاری دیتابیس کافی نیست.

شواهد محلی

۴۱ تست پروژه موفق شدند. Docker build روی Linux container واقعی در Docker Desktop موفق بود. smoke test اجرای migration، پاسخ سلامت، پیام فارسی، UID غیر root و نبودن polib و httpx در runtime را تأیید کرد. کانتینر آزمایشی پس از آزمون حذف شد. این نتیجه ممیزی مستقل امنیت یا آزمون بار نیست. وضعیت اجرای آنلاین workflow در صفحهٔ Actions همان commit قابل بررسی است.

منابع رسمی، بررسی‌شده در 12-10-2026

[۱] https://docs.docker.com/build/building/multi-stage/
[۲] https://docs.docker.com/build/building/best-practices/
[۳] https://docs.github.com/en/actions/tutorials/publish-packages/publish-docker-images
[۴] https://docs.github.com/en/packages/managing-github-packages-using-github-actions-workflows/publishing-and-installing-a-package-with-github-actions